

chatgpt_expand.py — ChatGPT/DeepSeek Expand Pipeline

Landscape discovery + Coherent Quadrant, one script. Works for ANY market.

START: `python scripts/run_chatgpt_expand.py --query "<Market>" --country <global|Country> --fresh`

1/6 ChatGPT Recall

Asks the LLM (paged, ~35 companies/page) to list companies it already knows in this market — cheap first pass before any live web search.

2/6 OpenAI SDK Web Search — Find Companies

Live web_search tool calls discover companies not in the LLM's own knowledge. Includes sub-step 2g below.

2g

Google AI List Discover (optional) — queries the local Google AI Overview scraper backend (<http://127.0.0.1:15561>, Chrome extension relay) for company-list-style AI Overviews, LLM-extracts company names from the scraped text. Needs the backend running + `GOOGLE_AI_SCRAPER_ENABLED=true`, else silently skipped.

3/6 DDGS / SearXNG Harvest + Extract (optional)

Optional supplementary search-engine harvest, LLM-extracts company names from results.

4/6 ChatGPT Verify (mid)

LLM checks each candidate against strict market-fit criteria (`verify_criteria_prompt`) and drops resellers/media/associations/geo-junk before the expensive fill step.

5/6 Fill Gaps to Target

Two-part fill, run in order, until the target row count is reached:

5a

Search-stack fill — Google AI + DDGS + SearXNG + Exa + Wikipedia + Owler (default `fill_backend=search`).

5b

OpenAI SDK residual fill — LLM fills remaining gaps directly.

6/6 ChatGPT Final Verify + Trim to Target

Final market-fit pass on the merged candidate set, then trims to the target row count (default 1000), ranked by data confidence.

6c Coherent Quadrant X / Y / Overall Scoring

Runs the SAME scoring machinery as `synthesize.py`, in this order:

chatgpt_expand.py — ChatGPT/DeepSeek Expand Pipeline

Landscape discovery + Coherent Quadrant, one script. Works for ANY market.

6c.1 Player-Type Filter — `classify_player_type()` decides hardware / software_service / consumer for this market, then DROPS every row whose role does not match: hardware -> Manufacturer only; software_service -> Solution Developer / Service Provider / System Integrator only; consumer -> Brand / Marketer only. This filtered set is what everything downstream (scoring, Excel, Quadrant chart) is built from.

6c.2 Define Market Axes — same fixed axis titles as `run_pipeline.py`: X = 'Product Strength', Y = 'Business Strategy', with 5 market-specific parameters per axis.

6c.3 Deep Crawl + Score — `smart_crawl` each surviving company, then LLM answers the generated scoring questions per company; computes X / Y / Overall and assigns quadrant by absolute median split.



6d Company Details Mapping

Builds the Brand | Company | Role | Quadrant | X | Y | Overall | Found-in table via `brand_display_fields()` — Company column is never blank: shows the company's own name, plus '(acquired by Parent)' when owned.



7 Write Final Outputs

Writes the final Excel (landscape) and the HTML/CSV Quadrant report — BOTH built from the same player-type-filtered row set from step 6c.1, so a hardware market's Excel and chart are Manufacturer-only end to end.



END: Final Excel (landscape) + Quadrant JSON/CSV/HTML (chart)

Every step is checkpointed (ExpandCheckpoint) — re-running without --fresh resumes from the last completed step instead of starting over.